AZURE DEVOPS DASHBOARDS

Complete Guide — All Widget Types with Use Cases

Work Tracking • Sprint Widgets • Charts • Test Results • Pipelines • Info Widgets

Practical Project: Online Banking App | Version 1.0 | 2026

Table of Contents

Table of Contents 2
1. What is a Dashboard in Azure DevOps? 3
  1.1 Key Dashboard Facts 3
  1.2 Dashboard vs Board vs Query 3
2. How to Create and Manage a Dashboard 4
  2.1 Creating a New Dashboard 4
  2.2 Adding Widgets to a Dashboard 4
  2.3 Dashboard Permissions 5
3. All Widget Types — Complete Reference 6
  Category 1: Work Tracking Widgets 6
  Category 2: Sprint & Progress Widgets 6
  Category 3: Chart Widgets 7
  Category 4: Test & Quality Widgets 8
  Category 5: Build & Pipeline Widgets 9
  Category 6: Informational Widgets 9
4. Recommended Dashboard Layouts by Role 11
  4.1 Developer Daily Dashboard 11
  4.2 Product Owner / Manager Dashboard 11
  4.3 QA / Test Dashboard 11
5. Connecting Queries to Dashboard Widgets 13
  5.1 Step-by-Step: Connect a Query to a Widget 13
  5.2 Most Useful Queries for Dashboard Widgets 13
6. Understanding the Sprint Burndown Chart 15
  6.1 Four Lines/Elements Explained 15
  6.2 Reading the Chart — Scenarios 15
7. Advanced Dashboard Tips & Best Practices 17
  7.1 Dashboard Organization Tips 17
  7.2 Widget Sizing Guide 17
  7.3 Common Mistakes to Avoid 17
  7.4 Quick Reference — All Widgets at a Glance 18

1. What is a Dashboard in Azure DevOps?

A Dashboard is a fully customizable web page inside your Azure DevOps project where you pin Widgets — small visual panels that display live data from your project. Dashboards are your team's real-time command center. Instead of navigating through multiple pages to check sprint progress, bug counts, build status, and test results, you see everything on a single screen.

Simple Definition

Dashboard = A collection of Widgets arranged on one page. Each Widget pulls live data from your project and displays it visually — as a number, chart, table, or burndown graph. The data refreshes automatically — no manual updates needed.

1.1 Key Dashboard Facts

- You can create multiple dashboards in one project — one for developers, one for managers, one for QA
- Each dashboard is owned by a team. Team dashboards are visible to everyone on that team.
- You can also create personal dashboards — visible only to you
- Dashboards auto-refresh every few minutes. You can also manually refresh anytime.
- Widgets are linked to live project data — Shared Queries, pipelines, sprints, test runs
- Dashboard permissions are controlled in Project Settings > Dashboards

1.2 Dashboard vs Board vs Query

Tool	Purpose & When to Use
Dashboard	Monitoring view. Shows key metrics, charts, and summaries at a glance. Used by managers, POs, and team leads every morning. Read-only — you cannot update items from here.
Board	Working view. Drag cards to update status during daily standup. Used by developers throughout the day. Interactive — dragging a card updates the work item state.
Query	Search view. Finds specific items matching filter conditions. Used for reporting, bulk updates, and feeding Dashboard widgets. Saves as reusable searches.

2. How to Create and Manage a Dashboard

2.1 Creating a New Dashboard

1 Navigate to Dashboards
In Azure DevOps, click the 'Overview' section in the left sidebar. Then click 'Dashboards'. You will see any existing dashboards listed here.

2 Create New Dashboard
Click '+ New Dashboard' button at the top right. A dialog box appears asking for the dashboard name and owner team.

3 Configure Settings
Enter a Name (e.g. 'Sprint 3 Team Dashboard'). Select the Team that owns it (e.g. 'Online Banking Team'). Choose visibility: Team dashboard (shared) or Personal dashboard (private only to you).

4 Dashboard is Created
Click 'Create'. An empty dashboard grid appears. You are now in edit mode and can start adding widgets.

2.2 Adding Widgets to a Dashboard

1 Enter Edit Mode
Click the pencil icon (Edit) at the top right of any dashboard. The dashboard enters edit mode — you see a grid with dashed borders and a '+' widget panel on the right.

2 Browse the Widget Catalog
Click '+ Add Widget'. A side panel opens showing all available widgets. You can search by name (e.g. type 'burndown' to find the Sprint Burndown widget).

3 Add the Widget
Click 'Add' next to any widget. It appears on the dashboard as a placeholder tile. You can drag it to any position on the grid.

4 Configure the Widget
Click the gear icon on the widget to open its settings. Most widgets need you to select a query, sprint, pipeline, or test plan as their data source. Fill in the required fields and click 'Save'.

5 **Save the Dashboard**
Click 'Done Editing' to save your changes and exit edit mode. The dashboard is now live and will refresh automatically.

2.3 Dashboard Permissions

Permissions are set in Project Settings > Dashboards. Three key settings:

Permission	What it Controls
Create dashboards	Allow team members to create new dashboards. Enabled by default for all contributors.
Edit dashboards	Allow team members to add, move, and configure widgets on team dashboards.
Delete dashboards	Allow team members to delete dashboards. Typically restricted to admins only.

3. All Widget Types — Complete Reference

Azure DevOps organizes widgets into six categories. Each widget below includes a full description, the best use case, and configuration instructions.

Category 1: Work Tracking Widgets

These widgets monitor work items — PBIs, Bugs, Tasks, Features. They answer the question: 'What work exists and who is doing it?'

WORK ITEMS
Assigned to Me

Displays a list of all work items currently assigned to the logged-in user, across all sprints and states. Shows title, state, and work item type.

Use case: *A developer opens the dashboard every morning and immediately sees their 4 active tasks and 1 critical bug without searching.*

How to configure: No configuration needed. Automatically shows items for whoever is viewing the dashboard. Add this to every developer's personal dashboard.

WORK ITEMS
Work Items

Shows a count of work items matching a saved query as a large number. Can be configured to turn red when the count exceeds a threshold, creating an alert.

Use case: *Manager sees a tile showing '7' active bugs. When it exceeds 10 it turns red — automatic alert without anyone sending an email.*

How to configure: Configuration: Select a Shared Query as the data source. Optionally set a threshold value and a warning color (yellow) and critical color (red).

WORK ITEMS
Query Results

Displays the full results of a saved query as a scrollable table, showing columns like Title, State, Assigned To, Story Points, and Priority.

Use case: *Show all active items in the current sprint as a live list. Team members can click any row to open the work item directly.*

How to configure: Configuration: Select a Shared Query. Choose which columns to show. Set number of rows to display (max 200). Sort by a field.

WORK ITEMS
Query Tile

Compact tile showing the count from a query as a bold number with a label. Smaller than 'Work Items' widget, useful for dashboards with many metrics.

Use case: *Create a 2x2 grid of tiles: 'Active Bugs: 7', 'Unestimated Items: 3', 'Open PRs: 5', 'Failed Builds: 1'. Quick health check at a glance.*

How to configure: Configuration: Select a Shared Query. Add a title label. Set size (1x1 is smallest). Optionally add a hyperlink to the full query results.

Category 2: Sprint & Progress Widgets

These widgets track sprint health, team velocity, and workload distribution. They answer: 'Are we on track to finish the sprint?'

SPRINT
Sprint Burndown

A line chart showing the remaining work (hours or story points) decreasing day-by-day through the sprint. Shows the Remaining line, Total Scope line, Remaining Capacity (dashed), and Ideal Trend. When remaining work is ABOVE the ideal trend line, you are behind schedule.

Use case: *During the daily standup, the Scrum Master opens the Sprint Burndown to see if the team is burning down work fast enough to finish by the sprint end date.*

How to configure: Configuration: Select Team, Iteration (sprint), and what to measure (Remaining Work in hours OR Story Points). The chart auto-populates as tasks are updated.

SPRINT
Sprint Overview

A compact summary widget showing: sprint name, start/end dates, a progress bar (% complete), total story points, completed points, and remaining points.

Use case: *A quick 'at a glance' status: 'Sprint 3: 63% complete, 12 of 19 story points done, 6 days remaining.' Perfect for the top of a team dashboard.*

How to configure: Configuration: Select Team and Iteration. Choose display format (bar chart or numerical). Optionally show days remaining countdown.

SPRINT
Sprint Capacity

A bar chart showing each team member's Remaining Work (how much they still have to do) versus their available Capacity (hours they have left in the sprint). Red bars mean a person is over-allocated.

Use case: *During sprint planning, spot that Ravi has 20 hours of remaining work but only 12 hours of capacity left. Redistribute work before it becomes a problem.*

How to configure: Configuration: Select Team and Iteration. Ensure team members have set their capacity in the Sprint Capacity tab (Boards > Sprints > [Sprint] > Capacity).

SPRINT
Velocity

A bar chart showing planned vs. completed story points for the last N sprints side by side. Each sprint has two bars: what was committed and what was actually delivered. The average velocity is shown as a horizontal line.

Use case: *During sprint planning, review the velocity chart to see the team averages 38 points per sprint. Use this to decide how many points to commit to in the next sprint.*

How to configure: Configuration: Select Team. Set number of sprints to show (typically 6). Choose between story points or work items count.

Category 3: Chart Widgets

These widgets visualize data from queries as charts. They answer: 'What does the distribution of our work look like?'

CHART
Chart for Work Items

Renders the results of any saved query as a chart. Supports Pie chart, Bar chart, Stacked bar, Column, Pivot table, and Area chart. You configure the X and Y axes and choose the chart type.

Use case: *Pie chart of all PBIs by State (New=blue, Active=green, Done=gray). Immediately shows if too many items are stuck in Active.*

How to configure: Configuration: Select Shared Query. Choose chart type. Set Group by field (e.g. State, Assigned To, Priority, Area). Add title. Size: min 2x2 tiles.

CHART
Cumulative Flow Diagram (CFD)

An area chart where each colored band represents a workflow state (New, Active, Resolved, Done). Band width shows how many items are in that state each day. A widening 'Active' band signals a bottleneck — work is piling up there.

Use case: *Team notices the 'In Review' band keeps growing wider over 5 days. This reveals code reviews are a bottleneck — too much waiting to be reviewed.*

How to configure: Configuration: Select Team and Iteration (or date range). Select which states to include. Each state gets a color band automatically.

CHART
Lead Time & Cycle Time

A scatter plot showing how long individual work items took from creation (Lead Time) or from first activation (Cycle Time) to completion. Outlier dots above the trend line represent items that took unusually long.

Use case: *The team sees that bugs average 8 days to resolve. Two outlier bugs took 22 days. These are investigated to find what caused the delay.*

How to configure: Configuration: Select Team. Set date range. Select work item types. View by Lead Time or Cycle Time. 50th/85th/95th percentile lines are drawn automatically.

CHART
Deployment Status

Shows the latest deployment result for each environment (Dev, QA, Staging, Production) with colored status icons: green checkmark = success, red X = failed, orange clock = in progress.

Use case: *Team opens the morning dashboard and sees Production has a red X — the overnight deployment failed. They immediately investigate without waiting for an alert email.*

How to configure: Configuration: Select a Release Pipeline. The widget auto-reads the pipeline's environments and shows the latest status for each stage.

Category 4: Test & Quality Widgets

These widgets track QA results and code quality. They answer: 'Is our product working as expected?'

TEST
Test Results Trend

A line chart showing pass, fail, and skipped test counts across multiple test runs over time. Shows whether your pass rate is improving or declining sprint over sprint.

Use case: *After Sprint 3, the pass rate dropped from 94% to 78%. This reveals a regression — new code broke existing tests. The team investigates immediately.*

How to configure: Configuration: Select a Test Plan or Test Run filter. Set date range (typically current sprint or last 30 days). Choose to show pass%, fail%, or counts.

TEST
Requirements Quality

For each PBI (requirement), shows: how many test cases are linked, how many passed, how many failed, and how many have not been run. Gives QA a PBI-by-PBI quality view.

Use case: *QA Manager sees 'Transfer money to account' PBI has 8 test cases: 5 Pass, 2 Fail, 1 Not Run. The PBI is not ready to ship — 2 tests still failing.*

How to configure: Configuration: Select Test Plan. Select PBIs to show. The widget reads test case links on each PBI and aggregates the latest test run results.

Category 5: Build & Pipeline Widgets

These widgets monitor CI/CD health. They answer: 'Is our code building and deploying successfully?'

PIPELINE
Build History

A bar chart showing recent pipeline build results over time. Each bar represents one build: green = success, red = failure, yellow = partially succeeded. Shows your pipeline's reliability trend.

Use case: *The main branch CI pipeline failed 3 of the last 5 builds. This is a warning sign — the team investigates who broke the build and why it keeps failing.*

How to configure: Configuration: Select Build Pipeline definition. Set number of builds to show (default 30). You can filter by branch (e.g. only show 'main' branch builds).

PIPELINE
Release Pipeline Overview

Shows all your release pipelines in a card layout. Each card shows the pipeline name, latest release, and status of each deployment stage (Dev, QA, Production) with pass/fail indicators.

Use case: *At a glance: Dev environment is green (deployed 2 hours ago), QA is yellow (pending manual approval), Production is green (last deployed 3 days ago).*

How to configure: Configuration: Select one or more Release Pipelines to display. Optionally show the deployed artifact version number on each environment card.

Category 6: Informational Widgets

These widgets provide context, navigation, and team information. They answer: 'What does the team need to know right now?'

INFO
Markdown

A free-text widget where you write formatted Markdown. Use it for sprint goals, team announcements, important links, meeting notes, or instructions. Supports headings, bold text, bullet points, and hyperlinks.

Use case: *Pin the sprint goal at the top of the dashboard: 'Sprint 3 Goal: Complete Fund Transfer Module with zero Priority 1 bugs by Feb 11.'*

How to configure: Configuration: Click the widget, write Markdown content. Supports # headings, **bold**, *italic*, - bullet points, and [hyperlinks](url). Resize to fit content.

INFO
Links

A panel of quick-access hyperlinks. Add links to external tools, documentation, wiki pages, Figma designs, Confluence, Slack channels, or any URL. Saves time from bookmarking.

Use case: *Links panel titled 'Team Resources' with links to: API Design Document, Figma Prototype, Deployment Runbook, Team Slack Channel, SonarQube Report.*

How to configure: Configuration: Add as many links as needed. Each link has a title and URL. You can group links under section headings and reorder them by drag.

INFO **Embedded Webpage**	Embeds any external web URL as an iframe directly inside the dashboard. Works with Grafana, SonarQube, Power BI dashboards, Kibana, or any public URL that allows embedding. **Use case:** *Embed the Grafana production monitoring dashboard inside Azure DevOps. Engineers see application performance metrics right next to their sprint progress.* **How to configure:** Configuration: Enter the full URL to embed. Set the widget size (large recommended). Note: The external page must allow X-Frame-Options embedding.

INFO **Team Members**	Displays all current team members with their profile pictures, display names, and roles. Each avatar is clickable, opening the member's Azure DevOps profile. **Use case:** *New team joiners open the dashboard and immediately see all 8 team members, their names, and can click to find contact info. Great for onboarding.* **How to configure:** Configuration: Select the Team to display. No other configuration needed. Members automatically update when people are added or removed from the team.

4. Recommended Dashboard Layouts by Role

Different roles need to see different information. Here are three ready-to-use dashboard configurations for the Online Banking App project:

4.1 Developer Daily Dashboard

Purpose: Used every morning before standup. Shows what to work on today and whether the CI pipeline is healthy.

Widget	Configuration for Online Banking App
Assigned to Me	No config needed. Shows my active tasks and bugs automatically.
Sprint Overview	Team: Online Banking, Iteration: @CurrentIteration. Shows sprint % complete.
Sprint Burndown	Team: Online Banking, Iteration: Sprint 3, measure by Story Points.
Build History	Pipeline: OnlineBankingApp-CI, show last 20 builds for 'main' branch.
Query Results	Query: 'My active sprint items'. Columns: Title, State, Remaining Work.
Markdown	Content: Sprint 3 Goal and today's standup notes or blockers.

4.2 Product Owner / Manager Dashboard

Purpose: Planning and reporting view. Shows sprint progress, velocity trends, and overall work status.

Widget	Configuration for Online Banking App
Velocity	Team: Online Banking, last 6 sprints. Shows average velocity for planning.
Sprint Overview	Current sprint summary with points and days remaining.
Chart for Work Items	Query: All Sprint items. Chart type: Pie. Group by: State. Shows New/Active/Done split.
Query Tile	Query: 'Unestimated PBIs'. Threshold: 5 (turn red). Alert for grooming needed.
Cumulative Flow	Team: Online Banking, current sprint. Reveals workflow bottlenecks.
Deployment Status	Release pipeline: OnlineBankingApp-Release. Shows Dev/QA/Prod status.

4.3 QA / Test Dashboard

Purpose: Quality monitoring. Shows test results, active bugs, and release readiness.

Widget	Configuration for Online Banking App
Test Results Trend	Test Plan: Online Banking Sprint 3. Date range: last 14 days.
Requirements Quality	Test Plan: Sprint 3 Test Plan. Shows pass/fail per PBI.
Query Tile x3	Tile 1: 'Active P1 Bugs'. Tile 2: 'Active P2 Bugs'. Tile 3: 'Resolved Bugs'.
Query Results	Query: 'Critical bugs not resolved'. Columns: Title, Priority, Assigned To.
Build History	CI pipeline last 30 builds. QA checks if tests are consistently passing.
Deployment Status	Release pipeline across all environments. Pre-release readiness check.

5. Connecting Queries to Dashboard Widgets

The most powerful dashboard pattern is: Shared Query → Dashboard Widget. A properly set up Shared Query becomes a live data source that automatically updates every widget connected to it.

The Golden Rule

Any query used in a Dashboard Widget MUST be a Shared Query (not a My Query). My Queries are private and cannot be accessed by the widget system. Always save dashboard-backing queries in the Shared Queries folder.

5.1 Step-by-Step: Connect a Query to a Widget

1 Create the Shared Query
Go to Boards > Queries. Click '+ New Query'. Build your query (e.g. Type=Bug AND State=Active AND Iteration=@CurrentIteration). Click 'Save Query'. In the Save dialog, choose 'Shared Queries' folder. Name it clearly: 'Sprint Active Bugs'.

2 Go to Dashboard
Navigate to Overview > Dashboards > your team dashboard. Click Edit (pencil icon).

3 Add a Query Widget
Click '+ Add Widget'. Search for 'Query Results' or 'Query Tile' or 'Work Items'. Click 'Add'.

4 Configure with Your Query
Click the gear icon on the new widget. In the 'Query' dropdown, find your saved query 'Sprint Active Bugs' under Shared Queries. Select it. Click Save.

5 Done — Live Data!
The widget now shows live results from your query. Every time a bug is created or closed, the widget updates automatically. No manual refresh needed.

5.2 Most Useful Queries for Dashboard Widgets

Query Name	Clauses	Best Widget
Sprint active items	`Iteration=@CurrentIteration AND State=Active AND Type In (PBI, Bug)`	Query Results (list)
Active critical bugs	`Type=Bug AND Priority<=2 AND State<>Closed`	Query Tile (count with alert)

Unestimated PBIs	`Type=PBI AND Story Points=[empty] AND State<>Closed`	Query Tile (alert if >0)
My tasks today	`Assigned To=@Me AND State=Active AND Type=Task`	Assigned to Me or Query Results
Items done this sprint	`Iteration=@CurrentIteration AND State=Done AND Type In (PBI, Bug)`	Query Tile (completion count)
Blocked items	`Tags Contains 'blocked' AND State<>Closed`	Query Results (with Priority col)
PRs linked to sprint	`Work Item Type=Pull Request AND Iteration=@CurrentIteration`	Query Results
Sprint all items chart	`Iteration=@CurrentIteration AND Type In (PBI, Bug)`	Chart for Work Items (pie by state)

6. Understanding the Sprint Burndown Chart

The Sprint Burndown is the most important widget in Azure DevOps. It tells you at a glance whether your sprint is healthy or at risk. Here is a complete explanation of every element.

6.1 Four Lines/Elements Explained

Element	What it Represents & How to Read It
Blue Bars (Remaining)	The actual remaining work (hours or story points) each day. Should decrease steadily. If it stays flat for 2+ days, no work is being completed — investigate why.
Orange Line (Total Scope)	The total amount of work committed to the sprint. A flat line means no scope was added. If it goes UP mid-sprint, new work was added after the sprint started — this is 'scope creep' and should be discussed with the team.
Green Dashed Line (Remaining Capacity)	How many work hours the team still has available based on sprint calendar. Starts at full capacity and decreases as days pass, regardless of work done. If remaining work exceeds remaining capacity, the sprint is in trouble.
Gray Line (Ideal Trend)	A perfectly straight line from the starting total down to zero at the sprint end. This is what perfect burndown would look like. Compare the blue remaining bars to this line: above = behind pace, below = ahead of pace.

6.2 Reading the Chart — Scenarios

Scenario 1: Healthy Sprint

Blue bars are close to or slightly below the gray ideal trend line. Orange scope line is flat. Green capacity line is well above the blue remaining bars.
Action: No action needed. Continue current pace. Sprint will finish on time.

Scenario 2: At Risk Sprint

Blue bars are consistently ABOVE the gray ideal trend line by day 4 or 5 of the sprint. The gap keeps growing.
Action: Scrum Master calls an emergency discussion. Consider removing lower-priority items from the sprint to protect the most important ones.

Scenario 3: Scope Creep Detected

Orange Total Scope line goes UP mid-sprint (e.g. goes from 30 points to 38 points on day 5). This means items were added after sprint planning started.

Action: Review what was added. If critical, adjust capacity expectations. If not critical, move the new items to the backlog for next sprint.

7. Advanced Dashboard Tips & Best Practices

7.1 Dashboard Organization Tips

- Name dashboards clearly: 'Sprint 3 — Team Dashboard', 'Q1 Manager View', 'QA Release Dashboard'
- Create one dashboard per sprint for historical records — do not overwrite previous sprint dashboards
- Always put a Markdown widget at the top of every dashboard with the sprint goal and date
- Group similar widgets together: all sprint health widgets in the top row, all bug/quality widgets in the second row
- Use the Query Tile widget for critical metrics — it is more visually prominent than Query Results

7.2 Widget Sizing Guide

Widget	Recommended Size
Markdown (sprint goal)	Full width (4 columns wide)
Sprint Burndown	2 columns wide, 2 rows tall — needs space to show the chart clearly
Sprint Overview	1 column wide, 1 row tall — compact summary number
Velocity	2 columns wide, 2 rows tall — needs room for multi-sprint bars
Query Tile (count)	1 column wide, 1 row tall — just a number
Query Results (list)	2-3 columns wide, 2-3 rows tall — needs space for multiple rows
Chart for Work Items	2 columns wide, 2 rows tall — pie/bar chart needs space
Build History	2 columns wide, 1 row tall — horizontal bar chart fits here
Deployment Status	2 columns wide, 1-2 rows tall — depends on number of environments

7.3 Common Mistakes to Avoid

Mistakes Beginners Make With Dashboards

1. Using 'My Queries' as widget data sources — they do not work. Always use Shared Queries.
2. Not configuring Sprint Capacity before the sprint — the Burndown and Sprint Capacity widgets will show incorrect data.
3. Adding too many widgets — a cluttered dashboard is hard to read. Focus on 6-10 key widgets maximum.
4. Not setting iteration dates — Sprint Burndown and @CurrentIteration queries will not work without sprint start/end dates.

5. Making dashboards public when they contain sensitive data. Always check visibility settings match your team's needs.

7.4 Quick Reference — All Widgets at a Glance

Widget Name	Category	Data Source	Best Used By
Assigned to Me	Work	*Automatic*	Developers — daily personal view
Work Items	Work	*Shared Query*	Managers — count with threshold alert
Query Results	Work	*Shared Query*	Everyone — scrollable list of items
Query Tile	Work	*Shared Query*	Everyone — compact count tile
Sprint Burndown	Sprint	*Sprint/Iteration*	Scrum Master — daily sprint health
Sprint Overview	Sprint	*Sprint/Iteration*	Everyone — quick progress summary
Sprint Capacity	Sprint	*Sprint/Iteration*	Scrum Master — workload balance
Velocity	Sprint	*Sprint/Iteration*	Product Owner — planning & forecasting
Chart for Work Items	Chart	*Shared Query*	PO/Manager — distribution charts
Cumulative Flow	Chart	*Sprint/Team*	Scrum Master — bottleneck detection
Lead/Cycle Time	Chart	*Team/Dates*	Manager — delivery speed trends
Deployment Status	Chart	*Release Pipeline*	Dev/Ops — environment health
Test Results Trend	Test	*Test Plan/Run*	QA — quality trend over time
Requirements Quality	Test	*Test Plan*	QA — PBI-level test coverage
Build History	Pipeline	*Build Pipeline*	Developers — CI reliability
Release Pipeline Overview	Pipeline	*Release Pipeline*	DevOps — release stage status
Markdown	Info	*None (manual)*	Everyone — notes, goals, links
Links	Info	*None (manual)*	Everyone — quick navigation panel
Embedded Webpage	Info	*External URL*	Teams using Grafana, SonarQube
Team Members	Info	*Team*	New joiners — meet the team

— End of Azure DevOps Dashboards Guide —

Self-Guide to make a dashboard in Azure DevOps

Charts that can used to track status and trends

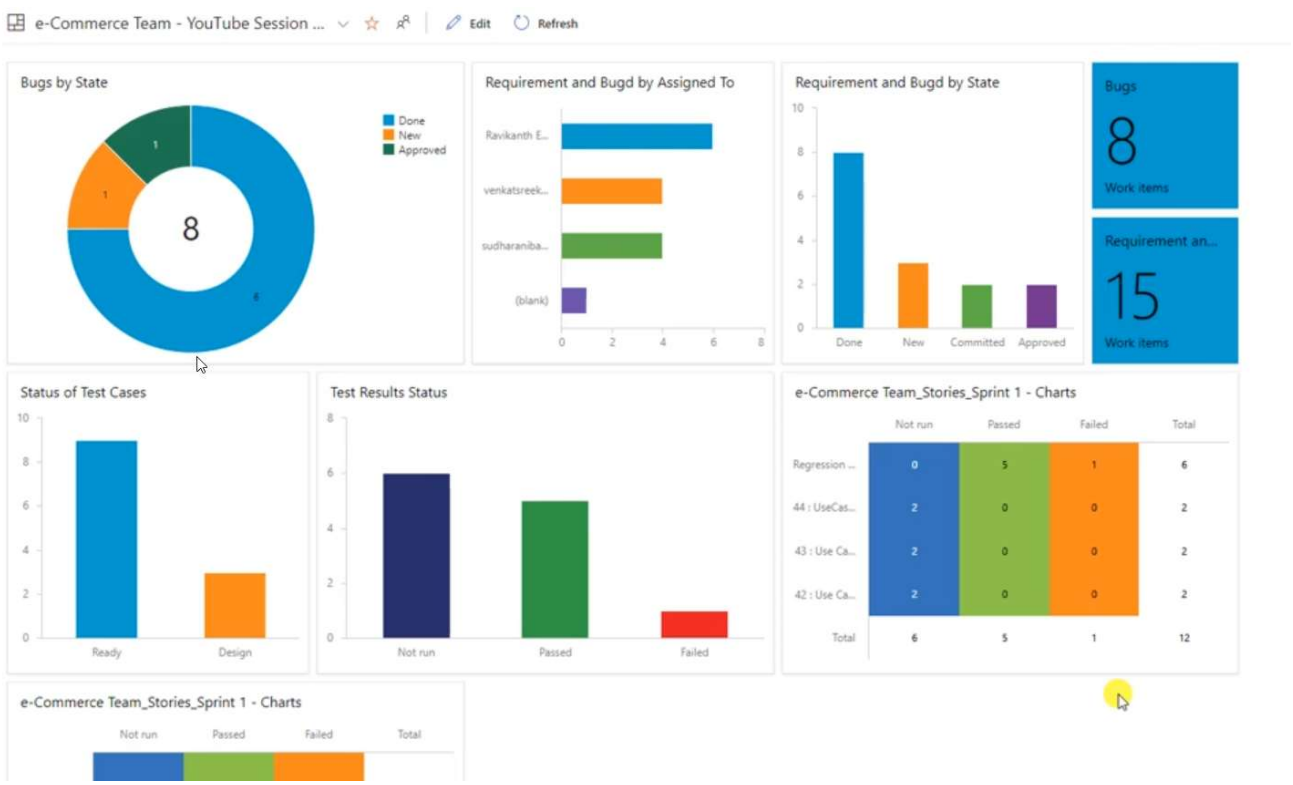
With flat-list queries, you can create various charts to monitor status, progress, and trends. To get started, you can open a shared query and create a chart based on your tracking interests.

Chart types include status—pie, bar, column, stacked bar, and pivot—and trend—stacked area, line, and area—charts.

Let me show how to create Shared Queries and Corresponding Charts

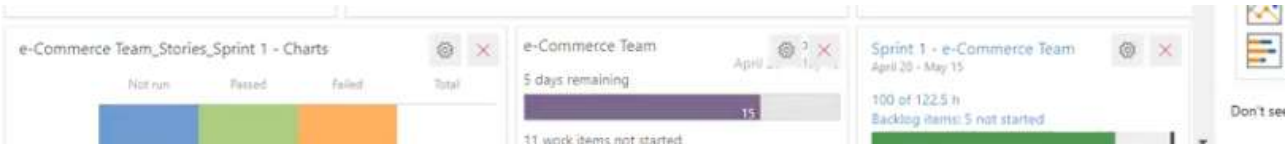
You can go for queries such as total number of bugs , total PBI or both in toal

- How many PBI or bugs assigened to one person
- Make a Graph for test cases , you can also use pivot table

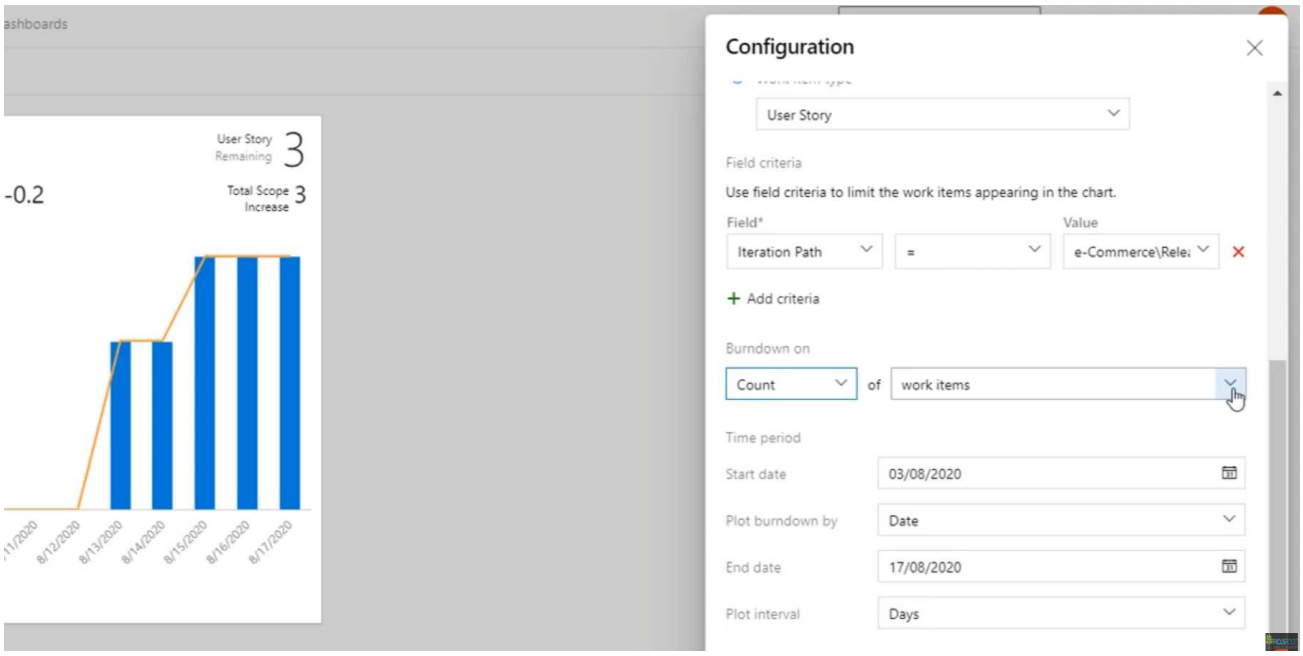


-

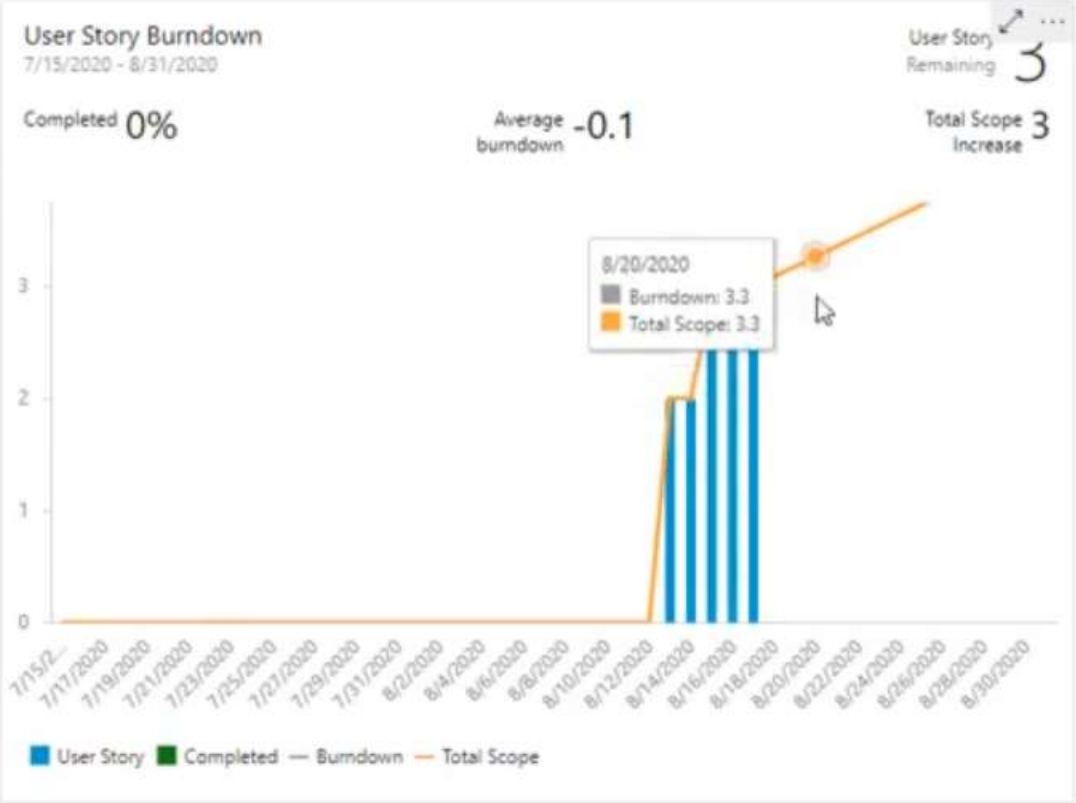
You can also add sprint overview charts , you can also use cumulative charts sprint overview , sprint capacity

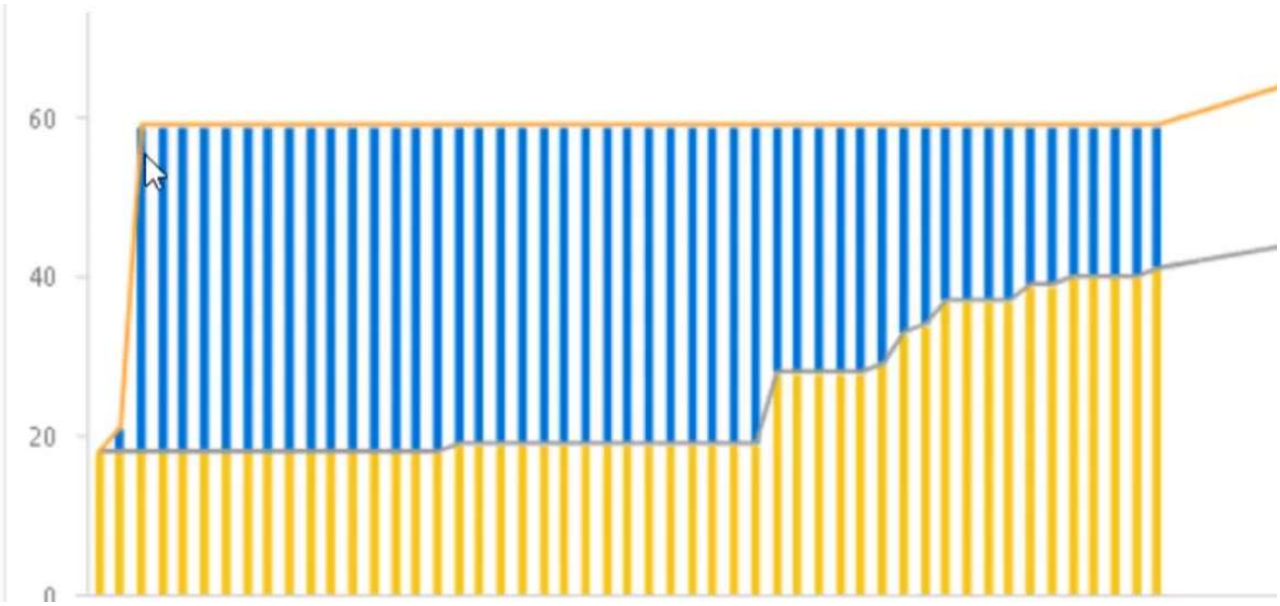


There are Burnups and Burn Downs charts



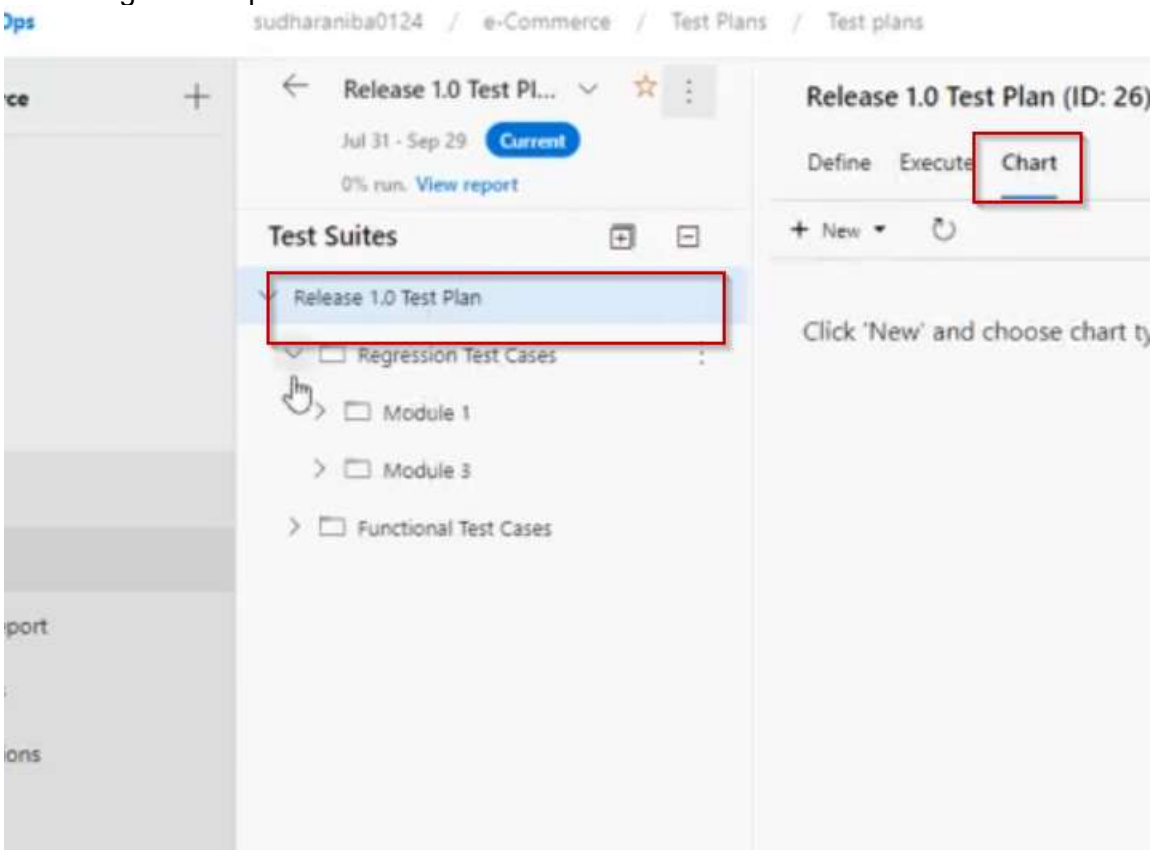
it means how many user stories are completed or remaning

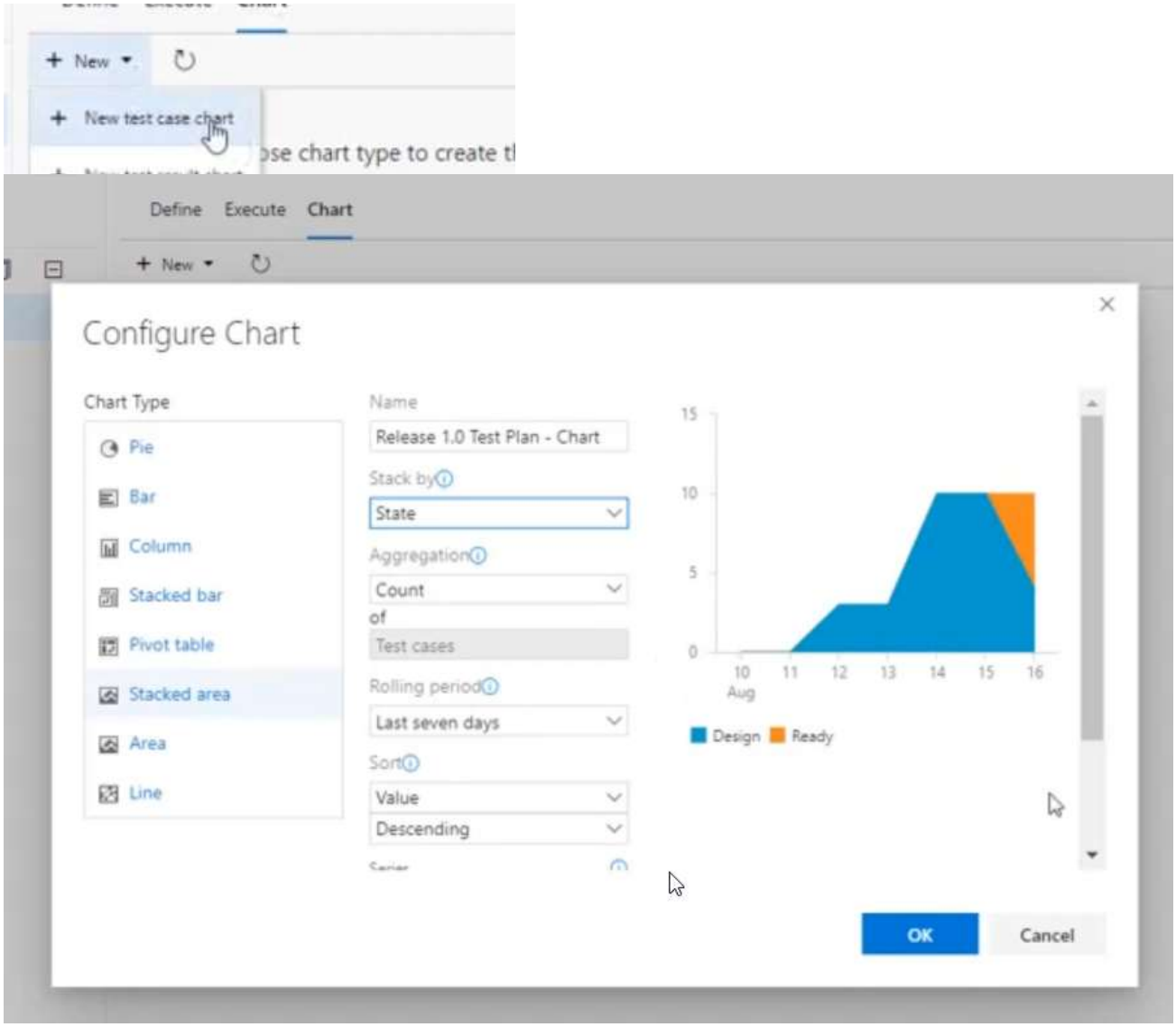




Burn up means the blue one is remaning , and yellow is completed and It is completed opposite for Burn down chart

You can go to test plan




New
New test case chart
Define
Execute
Chart
New
Configure Chart
Chart Type
Pie
Bar
Column
Stacked bar
Pivot table
Stacked area
Area
Line
Name
Release 1.0 Test Plan - Chart
Stack by
State
Aggregation
Count
of
Test cases
Rolling period
Last seven days
Sort
Value
Descending
15
10
5
0
10
Aug
11
12
13
14
15
16
Design
Ready
OK
Cancel